



실행방법만 정리

1-1. 체커보드 출력 & 촬영

- A4 30 mm 8×6(내부 코너 8×6) 체커보드 출력
링크: [Checkerboard-A4-30mm-8x6.pdf](#) (프린터 자동 맞춤/autofit 꺼두기)
- 여러 각도/거리에서 **2분 내외 영상** 촬영(코너 포함해 프레임 전체에 골고루 나오게)

1-2. 프레임 추출

```
1 mkdir -p frames
2 ffmpeg -i YOUR_VIDEO.mp4 -r 0.5 frames/frame_%04d.jpg
```

- **frames** → 원하는 폴더명으로 바뀌도 됨.
(예: `mkdir -p calib_imgs`)
- **YOUR_VIDEO.mp4** → 실제 영상 파일 이름/경로로 바뀌어야 함.
(예: `calib.mp4` 혹은 `./video/checkerboard.mp4`)
- **-r 0.5** → 추출 간격 (fps).
 - **0.5** = 2초에 1장
 - **1** = 1초에 1장
 - **2** = 1초에 2장 (0.5초마다 1장)
--> 영상 길이에 따라 적당히 조절
- **frames/frame_%04d.jpg** → 저장 경로와 파일 이름 패턴
 - **frames/** → 프레임 저장할 폴더 (위에서 만든 폴더와 같아야 함)
 - **frame_%04d.jpg** → **frame_0001.jpg**, **frame_0002.jpg** 처럼 4자리 번호 자동 증가
→ 폴더명/파일명은 바뀌도 되지만 보통 그대로 두는 게 편함

1-3. 내재 파라미터 계산

```
1 python compute_intrinsics.py \
2   --folder_images ./frames \
3   -ich 6 -icw 8 \
4   -s 30 -t 24 \
5   --debug
```

- `--folder_images ./frames`
→ `./frames` 를 프레임이 저장된 폴더 경로로 바꿔야 함.
(예: `--folder_images ./calib_frames`)
- `-ich 6 -icw 8`
→ 체커보드 코너 개수 (inner corners).
`-ich` : 세로 코너 개수
`-icw` : 가로 코너 개수
사용하는 체커보드 패턴에 맞게 꼭 바꿔야 함.
- `-s 30`
→ 체커보드 한 칸 크기 (mm 단위)
- `-t 24`
→ 최소 사용할 이미지 장수. 기본은 24장이면 충분.
👉 더 많이 쓰고 싶으면 늘려도 되고 그대로 뒤도 OK.
- `--debug`
→ 중간 과정(코너 검출 결과, reprojection error 등)을 확인할 수 있게 해줌.
👉 결과만 빠르게 뽑고 싶으면 빼도 됨.

`compute_intrinsics.py` 를 돌리면 나오는 것들

- `intrinsics.json`
카메라 내부 파라미터 저장 (K행렬, 왜곡계수 dist).
- **undistort 시각화 이미지**
보정 전/후 이미지가 저장됨 → 곡선이었던 직선(체커보드 줄)이 똑바르게 보이면 성공.
- **키포인트 재투영오차 로그**
체커보드 모서리를 탐지하고, 그 점들을 다시 3D → 2D로 투영했을 때의 오차.
→ 값이 작을수록 (1픽셀 이내) 보정 정확도가 좋다는 의미.

카메라 여러 개인 경우 `intrinsics`

```
1 # cam0
2 python compute_intrinsics.py --folder_images ./frames_cam0 -ich 6 -icw
  8 -s 30 -t 24 --debug
3
4 # cam1
5 python compute_intrinsics.py --folder_images ./frames_cam1 -ich 6 -icw
  8 -s 30 -t 24 --debug
```

이런 식으로 `intrinsics`는 개별적으로 여러번 실행해서 받아내면 됨